LAB 18

Subqueries and Solving Real Business Problems

SQL Topic: Subqueries in WHERE, Subqueries in SELECT, and Business Problem Solving With SQL

Estimated Time: 30–45 minutes **Difficulty:** Intermediate

Tool: MySQL Workbench, connected to the apex_retail database, with the orders and customers tables from Labs 04 and 13

A Note on This Lab

This lab teaches subqueries — a query nested inside another query. You'll learn how to use subqueries in **WHERE** to filter rows dynamically, and in **SELECT** to add calculated comparison columns — the same structured, guided approach used in every previous lab.

Subqueries: A Query Inside a Query

A **subquery** is a complete SELECT statement nested inside another query, always wrapped in its own parentheses. You've actually already met its simpler cousins — **IN** with a fixed list back in Lab 08 — a subquery just lets that list (or that single value) come from a **live query** instead of something you typed out by hand.

A subquery in **WHERE** filters rows using a value (or list of values) calculated on the fly, like `WHERE Total_Sales > (SELECT AVG(Total_Sales) FROM orders)`.

A subquery in **SELECT** adds a calculated column to every row, like showing the company-wide average right next to each order's own total — great for side-by-side comparisons.

About This Lab's Data

No new data to load. You'll be drawing on both **orders** (50 rows) and **customers** (50 rows) from Labs 04 and 13.

Part 1 — Learning the Syntax

1. Open **MySQL Workbench**, connect to your server, and type and run `USE apex_retail;`
2. Type and run the following query, which uses a subquery inside **WHERE**:

```
SELECT Order_ID, Total_Sales
FROM orders
WHERE Total_Sales > (SELECT AVG(Total_Sales) FROM orders);
```

Tip: the inner query **(SELECT AVG(Total_Sales) FROM orders)** runs first, calculates a single number (2323.48), and the outer query then filters using that number — exactly as if you'd typed WHERE Total_Sales > 2323.48 yourself.

3. Type and run the following query, which uses a subquery inside **SELECT** instead:

```
SELECT Order_ID, Total_Sales,
       (SELECT AVG(Total_Sales) FROM orders) AS OverallAvg
FROM orders
LIMIT 5;
```

Tip: this subquery calculates the same single number for **every** row — it doesn't filter anything, it just adds a comparison column so each order can be judged against the company average side by side.

4. Type and run the following query, which uses a subquery with **IN** to check the **customers** table against the **orders** table:

```
SELECT Customer_ID, Customer_Name
FROM customers
WHERE Customer_ID IN (SELECT Customer_ID FROM orders);
```

This finds the exact same 31 customers you found with an INNER JOIN back in Lab 14 — a subquery and a JOIN can often reach the identical answer by different routes.

Deliverable

Save your work as `Lab18_Subqueries_[YourName].sql`. The file should contain every query from today's lab, each with a one-line comment above it:

```
-- Confirms apex_retail is the active database
USE apex_retail;

-- Finds every order priced above the company-wide average using a
subquery in WHERE
SELECT Order_ID, Total_Sales
FROM orders
WHERE Total_Sales > (SELECT AVG(Total_Sales) FROM orders);

-- Adds the company-wide average as a comparison column using a subquery
in SELECT
SELECT Order_ID, Total_Sales,
       (SELECT AVG(Total_Sales) FROM orders) AS OverallAvg
FROM orders
LIMIT 5;

-- Finds customers who have placed at least one order using a subquery
with IN
SELECT Customer_ID, Customer_Name
```

```
FROM customers
WHERE Customer_ID IN (SELECT Customer_ID FROM orders);
```